# (Senior Academy)

## Your DevOps and Cloud Senior Partner




www.seniorsteps.net
https://www.facebook.com/seniorsteps.it
contact us: 0222700972 - 01090873748

**فرع مدينة نصر 1** : عمارة 4 – شارع محمد توفيق دياب – عباس العقاد - مدينة نصر – الدورال 1

**Senior Academy**
**www.seniorsteps.net**
**contact us: 0222700972 - 01090873748**
**شارع محمد توفيق دياب – عباس العقاد - مدينة نصر – الدورالأول**

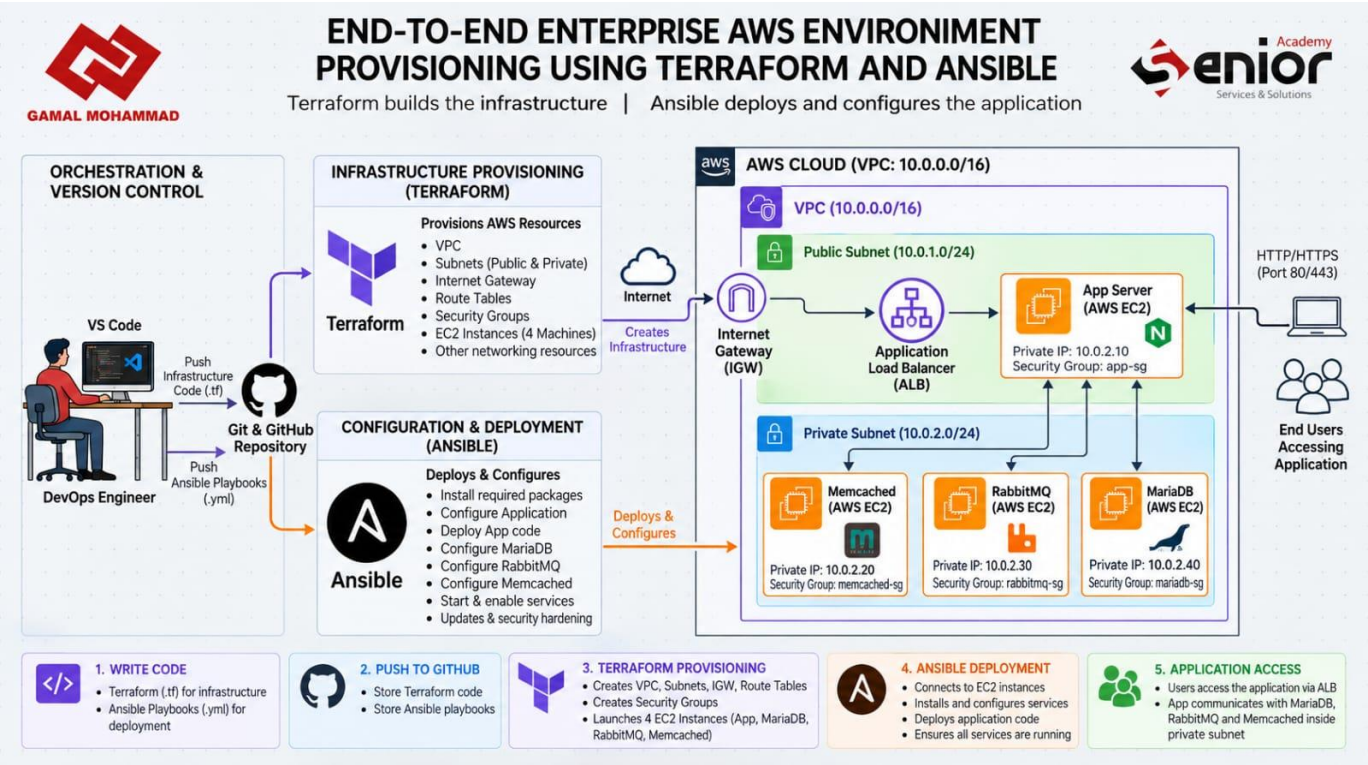
# *DevOps Workshop Advanced diploma*

# *25 projects*



# *Case Study Project 01*

## *End-to-End Enterprise AWS Environment Provisioning using Terraform and Ansible*




**Description:** A comprehensive hands-on project focused on combining Infrastructure as Code (IaC) with automated Configuration Management to build a complete, production-ready AWS environment for hosting scalable web applications. Engineers write declarative HashiCorp Configuration Language (HCL) scripts in Terraform to programmatically provision isolated AWS network topologies (VPCs, public/private subnets, internet gateways, and security groups) and launch Linux EC2 instances. Subsequently, Ansible playbooks are executed to automate OS-level configuration, install security updates, and deploy Nginx web servers across the newly provisioned instances, ensuring seamless system orchestration without manual intervention.

**Prerequisites:** Solid understanding of Linux system administration (Ubuntu/RedHat), core networking concepts (IP addressing, CIDR blocks, subnets, routing), basic familiarity with AWS cloud services (VPC, EC2, Security Groups), and foundational knowledge of shell scripting and Git version control workflows.

**Purpose:** To train DevOps engineers on bridging the gap between cloud resource provisioning and software

configuration management. By combining Terraform and Ansible, engineers learn how to eliminate manual server setup, enforce immutable configuration standards, prevent environment drift, and establish repeatable end-to-end infrastructure deployment pipelines.

**Tools Used:**

- HashiCorp Terraform
- Ansible
- AWS EC2
- Amazon VPC
- AWS Systems Manager (SSM)
- Nginx Web Server
- Linux (Ubuntu/RHEL)
- Git/GitHub
- Bash Shell Scripting.

**Knowledge Gained:**

- Architecting and deploying isolated AWS virtual private clouds (VPCs) with custom subnets, internet gateways, and route tables using Terraform.
- Provisioning and sizing AWS EC2 compute instances programmatically with defined security groups and key management.
- Writing modular Ansible inventory files and playbooks to automate Linux server hardening, package management, and service orchestration.
- Configuring and managing Nginx web servers dynamically to host multi-tier web applications.
- Integrating AWS Systems Manager (SSM) for secure, keyless remote server execution and management.
- Combining Infrastructure as Code (IaC) and Configuration Management paradigms into a single unified deployment workflow.
- Enforcing least-privilege security group policies and managing state synchronization across multi-tool environments.

**Functional & Technical Requirements**

1. **Infrastructure as Code (IaC) Layer – Terraform**

Toolchain: Use HashiCorp Terraform to define, provision, and manage the complete AWS infrastructure programmatically.

A. **Network Infrastructure**

- Create a custom AWS Virtual Private Cloud (VPC) with CIDR block "10.0.0.0/16".
- Create a Public Subnet with CIDR block "10.0.1.0/24".
- Create a Private Subnet with CIDR block "10.0.2.0/24".
- Provision and attach an Internet Gateway (IGW) to the VPC.
- Configure public and private Route Tables with appropriate routing rules.
- Configure outbound internet access for private instances through a NAT Gateway or an approved alternative when required for software installation and updates.

- Configure appropriate network isolation between public-facing and internal application components.

### B. EC2 Instance Provisioning

Terraform must automatically provision four Linux EC2 instances, each dedicated to a specific service.

**Instance 1 – Application Server**

- Host the web application.
- Install and configure Nginx through Ansible.
- Run the application and expose its required internal application port.
- Configure Nginx as a Reverse Proxy.
- Place the instance in the Public Subnet.

**Instance 2 – MariaDB Server**

- Host the MariaDB relational database.
- Store and manage application data.
- Accept database connections only from authorized application instances.
- Place the instance in the Private Subnet.

**Instance 3 – RabbitMQ Server**

- Host the RabbitMQ message broker.
- Enable asynchronous communication between application components.
- Restrict message broker access to authorized application instances.
- Place the instance in the Private Subnet.

**Instance 4 – Memcached Server**

- Host the Memcached caching service.
- Provide in-memory caching to improve application performance.
- Restrict cache access to authorized application instances.
- Place the instance in the Private Subnet.

### C. Security Groups

Terraform must create dedicated Security Groups for each EC2 instance.

**Application Security Group**

- Allow HTTP traffic on port 80 from the Application Load Balancer.
- Allow application-related internal traffic where required.
- Restrict administrative access.

**MariaDB Security Group**

- Allow TCP port 3306 only from the Application Server Security Group.

**RabbitMQ Security Group**

- Allow TCP port 5672 only from the Application Server Security Group.
- Restrict RabbitMQ management interface access.

**Memcached Security Group**

- Allow TCP port 11211 only from the Application Server Security Group.

**D. Load Balancing**

- Provision an AWS Application Load Balancer (ALB).
- Configure an HTTP listener on port 80.
- Create a Target Group for the Application Server.
- Register the Application EC2 instance with the Target Group.
- Configure health checks to monitor application availability.
- Allow external users to access the application through the ALB DNS name.

Note: The initial deployment uses one Application Server. The architecture should support future horizontal scaling by adding more application instances.

**2. Configuration Management & Deployment Layer – Ansible**

Toolchain: Use Ansible Playbooks (".yml") to automate operating system configuration, software installation, application deployment, and service management across all four EC2 instances.

**A. Inventory Management**

- Configure an Ansible inventory containing all four EC2 instances.
- Organize the inventory into dedicated groups:
- "app"
- "mariadb"
- "rabbitmq"
- "memcached"
- Support dynamic inventory using AWS EC2 instance metadata and tags.
- Ensure Ansible can securely connect to the target instances without requiring manual configuration.

**B. Operating System Configuration**

Ansible must perform the following tasks across all instances:

- Update system package repositories.
- Install required system packages.
- Apply available security updates.
- Configure required Linux services.
- Enable required services to start automatically after reboot.
- Apply basic operating system security hardening.

**C. Application Server Deployment**

Ansible must:

- Install Nginx.
- Install the required application runtime and dependencies.
- Deploy the application source code.
- Configure application environment variables.
- Configure Nginx as a Reverse Proxy.
- Configure the application to communicate with MariaDB, RabbitMQ, and Memcached.
- Enable and start the application service.
- Enable and start Nginx.
- Verify that the application is accessible through HTTP.

**D. MariaDB Deployment**

Ansible must:

- Install MariaDB Server.
- Configure MariaDB to listen on the required private network interface.
- Create the application database.
- Create a dedicated database user.
- Assign appropriate database privileges.
- Configure secure database authentication.
- Enable and start the MariaDB service.
- Verify database connectivity from the Application Server.

**E. RabbitMQ Deployment**

Ansible must:

- Install RabbitMQ Server and required dependencies.
- Configure the RabbitMQ service.
- Create an application-specific RabbitMQ user.
- Create the required Virtual Host.
- Assign appropriate permissions.
- Configure RabbitMQ to accept authorized connections.
- Enable and start the RabbitMQ service.
- Verify connectivity from the Application Server.

**F. Memcached Deployment**

Ansible must:

- Install Memcached.
- Configure Memcached to listen only on the private network interface.
- Configure the required memory allocation.
- Configure the service to use TCP port 11211.
- Enable and start the Memcached service.
- Verify cache connectivity from the Application Server.

**G. 2.7 Ansible Roles & Playbook Organization**

The Ansible project should use a modular structure with dedicated roles:

- "common" – Operating system updates and basic configuration.
- "app" – Application and Nginx deployment.
- "mariadb" – Database installation and configuration.
- "rabbitmq" – Message broker installation and configuration.
- "memcached" – Cache installation and configuration.

A main Ansible Playbook must orchestrate the deployment of all services.

**3. Application Architecture & Service Communication**

The deployed application must follow a multi-tier architecture.

**A. External Application Access**

- End users access the application through the public DNS name of the Application Load Balancer.
- The ALB forwards incoming HTTP requests to the Application Server.
- Nginx receives requests and forwards them to the application process.

**B. Database Communication**

- The Application Server connects to MariaDB using the database instance's private IP address or private DNS name.
- Database communication uses TCP port 3306.
- MariaDB must not be directly accessible from the public internet.

**C. Message Broker Communication**

- The Application Server connects to RabbitMQ through the private network.
- RabbitMQ uses TCP port 5672 for AMQP communication.
- RabbitMQ must not be directly accessible from the public internet.

**D. Caching Communication**

- The Application Server communicates with Memcached using TCP port 11211.
- Memcached provides temporary in-memory storage for application caching.
- Memcached must not be directly accessible from the public internet.

**E. Application Integration**

The deployed application must demonstrate functional integration with all three backend services:

- MariaDB for persistent data storage.
- RabbitMQ for asynchronous message processing.
- Memcached for application caching.

## 4. Infrastructure Provisioning & Deployment Workflow

The project must implement a clear separation between infrastructure provisioning and application configuration.

**Phase 1 – Infrastructure Provisioning (Terraform)**

1. Write Terraform configuration files using HCL.
2. Define the VPC, Subnets, Internet Gateway, Route Tables, and Security Groups.
3. Define the Application Load Balancer and Target Group.
4. Define the four EC2 instances.
5. Execute "terraform init".
6. Execute "terraform plan".
7. Execute "terraform apply".
8. Verify that all required AWS resources have been created successfully.

Expected Result: A complete AWS infrastructure containing four running EC2 instances and the required networking components.

**Phase 2 – Configuration & Deployment (Ansible)**

1. Obtain the provisioned EC2 instance information.
2. Generate or update the Ansible inventory.
3. Execute the main Ansible Playbook.
4. Apply operating system updates and basic security configuration.
5. Install and configure MariaDB.
6. Install and configure RabbitMQ.
7. Install and configure Memcached.
8. Deploy the application and configure Nginx.
9. Enable and start all required services.
10. Validate application connectivity to all backend services.

Expected Result: All four EC2 instances are configured automatically, with the application and its dependencies successfully deployed.

## 5. Acceptance Criteria & Validation

### A. Automated Infrastructure Provisioning

Running "terraform apply" must successfully provision:

- One custom VPC.
- One Public Subnet.
- One Private Subnet.
- One Internet Gateway.
- Required Route Tables and routing configurations.
- Dedicated Security Groups.

- Four Linux EC2 instances.
- One Application Load Balancer.
- One ALB Target Group.

Acceptance Criteria: All required AWS resources are created successfully without manual configuration through the AWS Console.

**B. Automated Service Deployment**

Running the main Ansible Playbook must:

- Configure all four EC2 instances.
- Install and configure Nginx and the application.
- Install and configure MariaDB.
- Install and configure RabbitMQ.
- Install and configure Memcached.
- Enable and start all required services.

Acceptance Criteria: All services are running successfully without manual software installation or configuration.

**C. Application Accessibility**

- Access the application through the ALB public DNS name.
- Verify that HTTP requests reach the Application Server.
- Verify that Nginx successfully forwards requests to the application.

Acceptance Criteria: The application interface is successfully displayed in a standard web browser.

**D. Backend Service Connectivity**

Verify the following:

MariaDB

- The application successfully connects to MariaDB.
- Database read and write operations execute successfully.

RabbitMQ

- The application successfully connects to RabbitMQ.
- A test message can be published and consumed.

Memcached

- The application successfully connects to Memcached.
- Cache values can be stored and retrieved.

Acceptance Criteria: All three backend services communicate successfully with the application through private network connections.

**E. Security Validation**

- MariaDB is accessible only from authorized application instances.
- RabbitMQ is accessible only from authorized application instances.
- Memcached is accessible only from authorized application instances.
- Backend instances do not have public IP addresses.
- Security Groups enforce the required access restrictions.

Acceptance Criteria: Internal services are isolated from direct public internet access.

**F. Idempotency & Repeatability**

- Re-running the Ansible Playbook must not introduce unnecessary configuration changes.
- Re-running "terraform plan" after successful provisioning must show no unexpected infrastructure changes.
- Infrastructure and application configurations must be maintained through version-controlled code.

Acceptance Criteria: The environment can be consistently reproduced and managed using Terraform and Ansible.

**You are Welcome**